QUALITY VERSUS STORAGE IN SVD-BASED IMAGE COMPRESSION: A MULTI DATASET ANALYSIS

PIERRE VISCONTI

Abstract. Low-rank matrix approximations using Singular Value Decomposition (SVD) are an effective approach to image compression. This paper examines how varying the number of top k singular values impacts image quality and file size across three diverse datasets of colored JPEG images. Each image, randomly sampled from their respective dataset, was reconstructed at multiple ranks and saved as a PNG file to avoid additional lossy compression. Quality was assessed using Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR), while file size was directly recorded from the output PNG file. Results show that MSE decreases exponentially with increasing k, PSNR grows logarithmically, and file size also follows a logarithmic trend due to PNG's compression algorithm. Based on our generated curves from all three datasets combined and each dataset individually analyzed, we determined the optimal k to be 25-30, which best balances image quality and file size.

1. Introduction

Low-rank matrix approximations are used in many different applications, such as compression, updating machine learning models to use fewer parameters, de-noising, and matrix completion problems [7]. This paper focuses on using low-rank matrix approximations for the compression of large matrices, specifically: image compression. According to Roughgarden and Valiant, in CS168 lecture notes on singular value decomposition at the University of Stanford, "low-rank approximations are a matrix analog to notions of dimensionality reduction for vectors." Since a "low-rank approximation provides a (lossy) compressed version of the matrix" [7] they are particularly useful for tasks such as machine learning, where image quality is not a priority since algorithms focus on pattern recognition rather than visual fidelity. Several different algorithms can be used for low-rank matrix approximations, such as Singular Value Decomposition (SVD) and Principal Component Analysis (PCA). We used SVD for this project which is later explained more rigorously in section 2.2. The goal in low-rank matrix approximations is to best approximate some given matrix A by a rank-k matrix, for a chosen k. In image compression, the image can be represented as a matrix of pixels intensities, where low rank approximations allow us to reconstruct the image using a subset of the singular values [7]. Since this compression is lossy in nature, there is a trade-off that exists bewteen the quality of the compressed image and the number of features left (or file size of the compressed image) which depends on the value of k. In this study we conduct a multi dataset analysis to address the question: how does varying the number of top k singular values we keep impact image quality and file size.

2. Methods and Computations

2.1. **Dataset.** For this project, a variety of different images were collected with the goal of generating results on a diverse dataset, rather than a dataset comprised of a single subject or topic.

We selected three different datasets comprised of colored images: "Landscape color and grayscale images" [5], "Butterfly Image Classification" [2], and, "100 Sports Image Classification" [4]. The first dataset is of landscape images, which was sourced from Kaggle with Bhabuk Ghimire listed as the owner but no other authorship given. This dataset contains 7129 colored landscape photos saved as JPEG file type, a form of lossy compression. We then also selected a dataset of close up butterfly photos, which was sourced from Kaggle with DePie listed as the owner but no other authorship given. The butterfly dataset contains 9,285 colored images of butterflies, all zoomed in to the point where the butterfly fills the entire frame, also saved as JPEG file type. The third dataset has images for 100 different sports, but we decided to select only the baseball images. The dataset contains 179 different colored images of baseball sporting images, such as players pitching or batting. This dataset is also comprised of JPEG images. This was sourced from Kaggle with Gerry listed as the owner but no other authorship given. These datasets were selected to have our results be more applicable to the real world, due to a diverse collection of images rather than one on a single subject. Additionally, we also have similar images within each dataset that are also different from one another which enables us to have repeated trials for each image subject.

2.2. **Singular Value Decomposition.** Suppose we have a matrix A that is of size $m \times n$, $A \in \mathbf{R}^{m\times n}$. Singal Value Decomposition (SVD) decomposes A into three operations.

(i) $$A = U\Sigma V^T$$

The three operations are as follows.

- $U \in \mathbf{R}^{m\times m}$ is a rotation in $\mathbf{R}^m$ whose columns are left singular vectors.
- $\Sigma \in \mathbf{R}^{m\times n}$ is a stretch (change dimension) with singular values on the diagonal.
- $V \in \mathbf{R}^{n\times n}$ is a rotation in $\mathbf{R}^n$ whose columns are right singular vectors.

U and V are orthogonal matrices and Σ is a diagonal matrix. The matrix U contains the orthonormal eigenvectors of AA^T, the matrix V contains the orthonormal eigenvectors of A^TA, and the matrix Σ contains the square roots of the eigenvalues of AA^T and A^TA. The singular values are denoted as

$$\sigma_1 \geq \sigma_2 \geq ... \geq \sigma_r > 0$$

where $\text{rank}(A) = r$ [1]. An equivalent expression to the factorization given by (i) is the following

(ii) $$A = \sum_{i=1}^{\min\{m,n\}} \sigma_i \vec{u_i} \vec{v_i}^T.$$

To approximate the matrix A by a rank-k matrix, which is known as low-rank approximation, we use the expression for A given by (ii) and only keep the first k terms on the right hand side, where we assume that the singular values have been sorted, as stated above mathematically. This can also be expressed in the following steps that depend on SVD in (i).

- Compute the SVD given by (i).
- "Keep only the top k left singular vectors: set U_k equal to the first k columns of U."
- "Keep only the top k right singular vectors: set V_k^T equal to the first k rows of V^T."
- "Keep only the top k singular values:" set Σ_k "equal to the first k rows and columns of" Σ, "corresponding to the k largest singular values of A.
- The rank-k approximation is then

$$A_k = U_k\Sigma_k V_k^T \quad [7].$$

2.3. **Presentation of Numerical Results.** To address the question: how does varying the number of top k singular values we keep impact image quality and file size, we present the following results which were obtained using our own MATLAB code. We had images from three separate datasets used in our analysis, which are all of JPEG file type. In order to keep the output images compressed by SVD from further lossy compression, we save the output images as a PNG rather than a JPEG. The file size of these output PNG images was measured using MATLAB as the variable k, defining the top k singular values we keep, was manipulated across a range of values. Rather than use a bias source for determining image quality, like a visual inspection from a human, we used two separate mathematical metrics called Mean Square Error (MSE) and Peak Signal to Noise Ratio (PSNR). MSE "is the averaged value of the square of the pixel-by-pixel difference between the original image and [compressed image]". It provides a "measure of the error produced in the cover image due to the data embedding process," where a lower value indicates a better quality embedding. PSNR "is the ratio between the maximum possible value of a signal and the power of distortion noise (MSE). It is measured in dB's" [3]. A higher PSNR value corresponds to a better quality compressed image [6].

We began by randomly sampling 50 images from each of the three datasets to eliminate bias and extend to results to the entire dataset. Each image was individually compressed using SVD for a specific k value and saved as a new PNG file. Then, for each image, the MSE and PSNR values were extracted in MATLAB using the built in functions, and the output PNG file size was recorded. The average (arithmetic mean) MSE, PSNR, and file size for all the images in a given dataset was then computed, as well as the overall average MSE, PSNR, and file size for all three datasets combined. This was performed for all k values ranging from 5 to 80 (in increments of 5) and then plotted using MATLAB. These results are displayed in the following graphs.

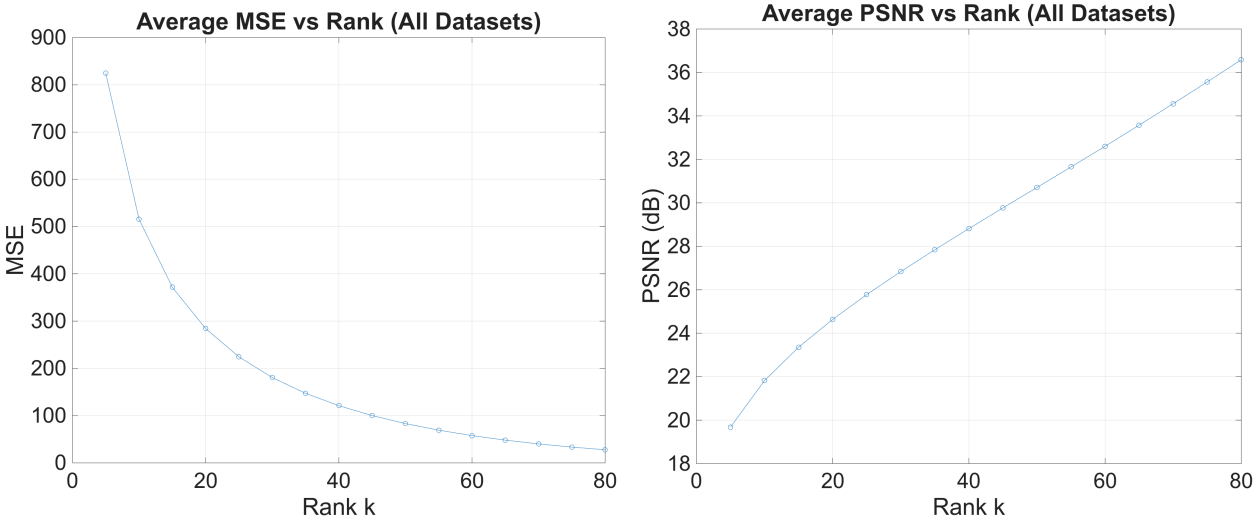

FIGURE 1. Average MSE and PSNR computed across all datasets.

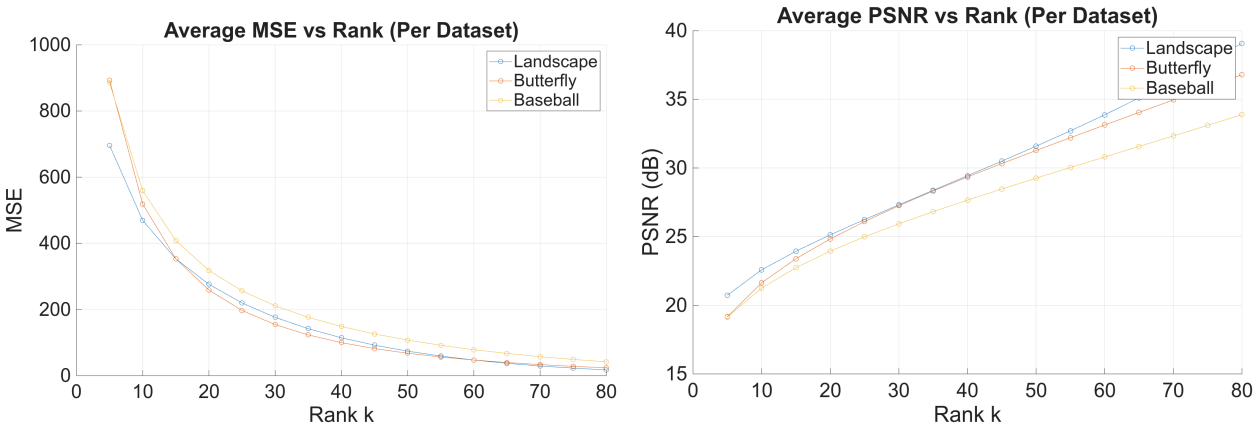

FIGURE 2. Average MSE and PSNR computed for each individual dataset.

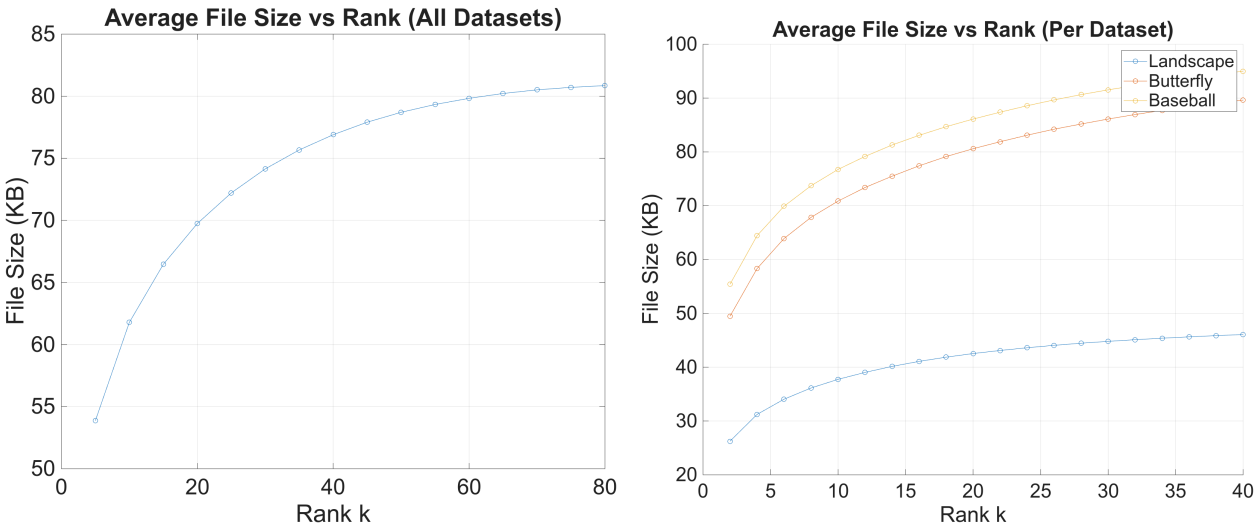

FIGURE 3. Average file size across all datasets and for each individual dataset.

2.4. **Assesment of Numerical Results.** We begin by analyzing the quality of the image versus the rank k or top k singular values that we keep. Recall that our quality metrics are MSE (where a lower value indicates higher quality) and PSNR (where a higher value indicates higher quality). Based on our results in Figure 1 and Figure 2, which plot the quality metrics versus rank k, MSE follows an exponential decay curve while PSNR follows logarithmic growth (this is more apparent later in Figure 5 where we plot a smaller range of k values). This means that quality increases at a higher rate while we increase k at the lower k values than when we increase k at higher k values. In other words, there are diminishing returns in quality as k is increased. Additionally, it is important to note that plotting the quality metrics individually per dataset yielded similar curves for each dataset, which means that the results hold for each one, see 2.

We next analyze the file size of our SVD compressed image versus the rank k or top k singular values that we keep. The plots for this are given in 3, where we plot the average file size versus rank for all the datasets combined, and for each dataset individually. Here we can see that the curves follow logarithmic growth. Even though the actual pixel count of the image is perfectly linear as k increases, due to the compression algorithms of PNG (and JPEG as well), the information

that is added by increasing k at lower values, is more easily compressible than that of higher k values, which yields a logarithmic curve. Additionally, the curves for each dataset individually are all similar to one another, despite their nominal values being different (due to different file sizes between datasets).

Finally, we answer the question, what is the optimal k value to balance quality and file size? To do this, we generated new graphs with k values ranging from 2 to 40, in increments of 2. We did this since higher k values provide diminishing returns, as seen in the earlier plots, so we needed to "zoom in" on lower k values to answer this question.

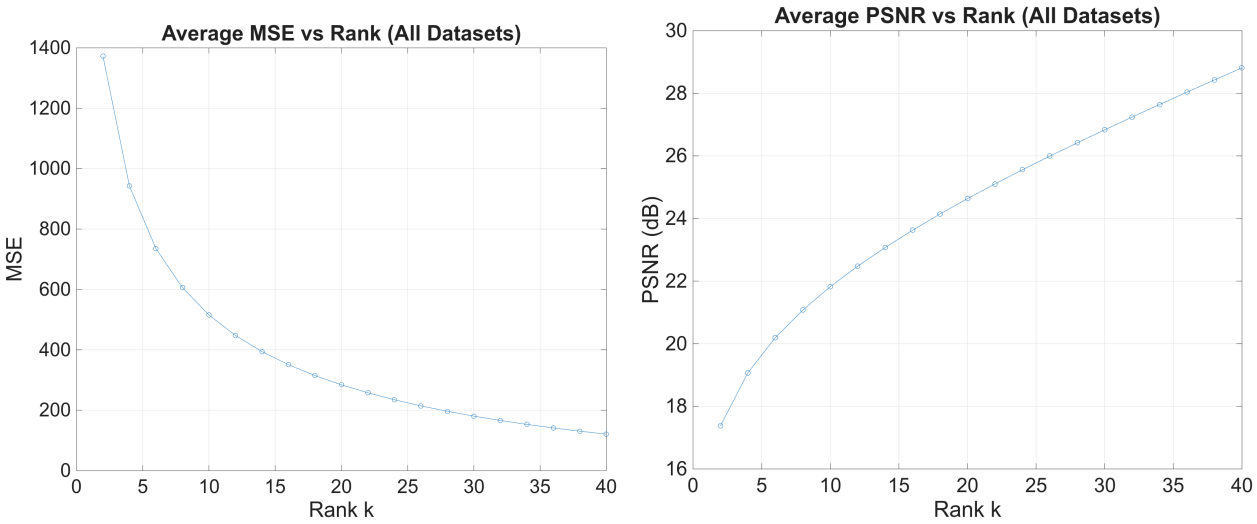

FIGURE 4. Average MSE and PSNR computed across all datasets.

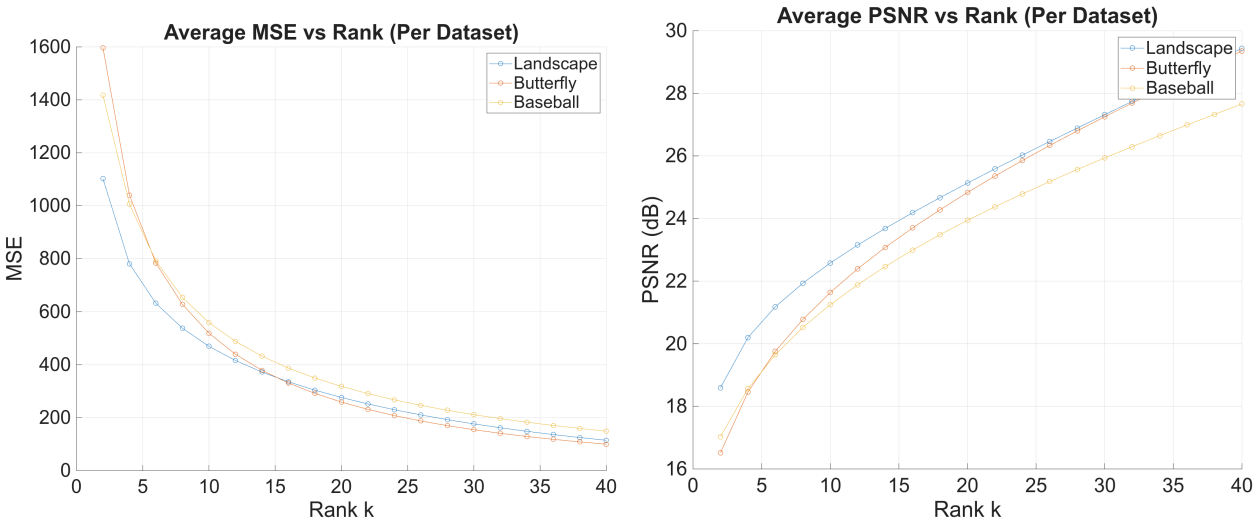

FIGURE 5. Average MSE and PSNR computed for each individual dataset.

We base our analysis on this section using a visual analysis of Figures 4 and 5 against Figure 3, with the goal of maximizing quality while reducing file size. A visual inspection is sufficient due to the subjective nature of the question "what is the optimal k?", since this differs based on individual

goals. We believe that a k value of roughly 25-30 provides the best balance of image quality and file size, while higher k values provide diminishing returns and longer run times. Although, if ones goal is compression for pattern recognition machine learning algorithms, we believe a lower k value could be used, and if ones goal is a task where image quality is more desirable, than a higher k value could be necessary.

3. Conclusions

In this paper we performed a multi-dataset analysis on three different colored image datasets to examine the relationship between image quality and file size versus the top k singular values that we keep in low rank image approximations using SVD. Images used for our analysis were randomly sampled from the datasets to eliminate bias, and the chosen datasets were selected since they represent different image subjects to better make our results applicable to the real world. To remove human bias, image quality was determined using two mathematical metrics, MSE and PSNR. File size was determined by recording the file size of the SVD compressed image when saved as a PNG file. We discovered that image quality follows exponential decay for MSE (where a lower value indicates higher quality), and logarithmic growth for PSNR (where a higher value indicates higher quality). This means that there are significant diminishing returns on quality as the variable k is increased. Additionally, the file size was found to follow logarithmic growth, despite the actual pixel count of the image being perfectly linear with k, due to PNG's compression algorithm. We determined the optimal k value to be around 25-30 to best balance image quality with file size, although lower and higher values could be chosen based on the desired workload.

References

[1] Breschine Cummins. SVD, October 7, 2025. Lecture notes, M441, Montana State University.

[2] DePie. Butterfly image classification. `https://www.kaggle.com/datasets/phucthaiv02/butterfly-image-classification`, 2021.

[3] GeeksforGeeks. Performance metrics for image steganography, 2025. Accessed December 4, 2025. URL: `https://www.geeksforgeeks.org/computer-networks/performance-metrics-for-image-steganography/`.

[4] Gerry. Sports image classification. `https://www.kaggle.com/datasets/gpiosenka/sports-classification`, 2021.

[5] Bhabuk Ghimire. Landscape color and grayscale images. `https://www.kaggle.com/datasets/theblackmamba31/landscape-image-colorization`, 2021.

[6] MathWorks. Psnr (peak signal-to-noise ratio), 2025. Accessed December 4, 2025. URL: `https://www.mathworks.com/help/vision/ref/psnr.html`.

[7] Tim Roughgarden and Gregory Valiant. CS168: The Modern Algorithmic Toolbox Lecture #9: The Singular Value Decomposition (SVD) and Low-Rank Matrix Approximations. Lecture notes, April 30, 2024. URL: `https://web.stanford.edu/class/cs168/l/l9.pdf`.

4. appendix

```
% Reduced SVD function
% k: vector of ranks to test, top k singular values
% num_samples: number of random images to sample from each dataset
function red_svd(k, num_samples)
    % Set random seed
    rng(441)
    % Define dataset folders
    dataset_paths = { 'landscape/color/', 'butterfly/', 'baseball/' };

    % Loop over all datasets
    for d = 1:length(dataset_paths)
        folder = dataset_paths{d};
        files = dir(fullfile(folder, '*.jpg'));
        num_files = length(files);
        % Randomly sample indices from the dataset
        rand_idx = randperm(num_files, min(num_samples, num_files));
        fprintf('\n=== Dataset: %s ===\n', folder);
        % Loop over sampled images
        for f = rand_idx
            filename = fullfile(folder, files(f).name);
            orig = imread(filename);
            Im = double(orig);
            fprintf('\nImage: %s\n', files(f).name);
            % Loop over ranks
            for r = k
                Imapprox = zeros(size(Im));
                for i = 1:size(Im,3)
                    [U,S,V] = svd(Im(:,:,i));
                    Imapprox(:,:,i) = U(:,1:r)*S(1:r,1:r)*V(:,1:r)';
                end
                approx_uint8 = uint8(Imapprox);
                % Save reduced (compressed) image as a PNG
                outFile = sprintf('output/%s_rank%d.png',
files(f).name(1:end-4), r);
                imwrite(approx_uint8, outFile);
                % Get the file size in KB
                fileInfo = dir(outFile);
                fileSizeKB = fileInfo.bytes / 1024;
                % Compute quality metrics
                mse_val = immse(approx_uint8, orig);
                psnr_val = psnr(approx_uint8, orig);
                % Print information
                fprintf('Rank %d: MSE=%.2f, PSNR=%.2f dB, PNG size=%.1f
KB\n', ...
                    r, mse_val, psnr_val, fileSizeKB);
            end
        end
    end
end
```

```
Not enough input arguments.

Error in red_svd (line 16)
```

```
rand_idx = randperm(num_files, min(num_samples, num_files));
                                   ^^^^^^^^^^^
```

Published with MATLAB® R2025a

```
% GENERATED BY BING COPILOT (FREE)
% Provided my red_svd() function to CoPilot and prompted it to create a
% function that plots the average values across all datasets for MSE, PSNR,
% and file size for a vector of k values. I then did a follow up prompt for
% it to plot the average values for each dataset seperately on the same
% graph.

function plot_results(k_values, num_samples)
    % plot_svd_tradeoff: Evaluate SVD compression trade-offs
    % k_values: vector of ranks to test
    % num_samples: number of random images to sample per dataset

    rng(441); % reproducibility
    dataset_paths = { 'landscape/color/', 'butterfly/', 'baseball/' };
    dataset_names = { 'Landscape', 'Butterfly', 'Baseball' };

    % Storage for results
    mse_all = zeros(length(k_values), length(dataset_paths));
    psnr_all = zeros(length(k_values), length(dataset_paths));
    size_all = zeros(length(k_values), length(dataset_paths));

    % Loop over datasets
    for d = 1:length(dataset_paths)
        folder = dataset_paths{d};
        files = dir(fullfile(folder, '*.jpg'));
        num_files = length(files);
        rand_idx = randperm(num_files, min(num_samples, num_files));

        mse_vals = zeros(length(k_values), length(rand_idx));
        psnr_vals = zeros(length(k_values), length(rand_idx));
        size_vals = zeros(length(k_values), length(rand_idx));

        % Loop over sampled images
        for f = 1:length(rand_idx)
            filename = fullfile(folder, files(rand_idx(f)).name);
            orig = imread(filename);
            Im = double(orig);

            % Loop over ranks
            for r = 1:length(k_values)
                k = k_values(r);
                Imapprox = zeros(size(Im));
                for i = 1:size(Im,3)
                    [U,S,V] = svd(Im(:,:,i));
                    Imapprox(:,:,i) = U(:,1:k)*S(1:k,1:k)*V(:,1:k)';
                end
                approx_uint8 = uint8(Imapprox);

                % Save temporarily to measure file size
                outFile = [tempname '.png'];
                imwrite(approx_uint8, outFile);
                fileInfo = dir(outFile);
```

```
            fileSizeKB = fileInfo.bytes / 1024;
            delete(outFile);

            % Compute metrics
            mse_vals(r,f) = immse(approx_uint8, orig);
            psnr_vals(r,f) = psnr(approx_uint8, orig);
            size_vals(r,f) = fileSizeKB;
        end
    end

    % Average across sampled images
    mse_all(:,d) = mean(mse_vals,2);
    psnr_all(:,d) = mean(psnr_vals,2);
    size_all(:,d) = mean(size_vals,2);
end

% Aggregate across datasets
mse_mean = mean(mse_all,2);
psnr_mean = mean(psnr_all,2);
size_mean = mean(size_all,2);

% === Aggregate plots ===
figure;
plot(k_values, mse_mean, '-o'); grid on;
xlabel('Rank k'); ylabel('MSE');
title('Average MSE vs Rank (All Datasets)');

figure;
plot(k_values, psnr_mean, '-o'); grid on;
xlabel('Rank k'); ylabel('PSNR (dB)');
title('Average PSNR vs Rank (All Datasets)');

figure;
plot(k_values, size_mean, '-o'); grid on;
xlabel('Rank k'); ylabel('File Size (KB)');
title('Average File Size vs Rank (All Datasets)');

% === Per-dataset plots ===
figure;
hold on;
for d = 1:length(dataset_paths)
    plot(k_values, mse_all(:,d), '-o', 'DisplayName', dataset_names{d});
end
grid on; xlabel('Rank k'); ylabel('MSE');
title('Average MSE vs Rank (Per Dataset)');
legend show; hold off;

figure;
hold on;
for d = 1:length(dataset_paths)
    plot(k_values, psnr_all(:,d), '-o', 'DisplayName', dataset_names{d});
end
grid on; xlabel('Rank k'); ylabel('PSNR (dB)');
title('Average PSNR vs Rank (Per Dataset)');
```

```
    legend show; hold off;

    figure;
    hold on;
    for d = 1:length(dataset_paths)
        plot(k_values, size_all(:,d), '-o', 'DisplayName', dataset_names{d});
    end
    grid on; xlabel('Rank k'); ylabel('File Size (KB)');
    title('Average File Size vs Rank (Per Dataset)');
    legend show; hold off;
end

Not enough input arguments.

Error in plot_results (line 18)
    mse_all = zeros(length(k_values), length(dataset_paths));
                           ^^^^^^^^
```

Published with MATLAB® R2025a